

BOOTCAMP · YNOV MONTPELLIER

Cours — Cybersécurité, Pentest & AB Testing

Support de référence — cybersécurité & data pour un parcours en
reconversion

Durée : 6 heures · Niveau : débutant (aucun prérequis technique)

Public : M1 en reconversion professionnelle

Date : 17/09/2026 · YNOV Campus Montpellier

Support de cours — à conserver

Cours — Cybersécurité, Pentest & AB Testing

Support de cours autoporteur, conçu pour un démarrage sans aucun prérequis en cybersécurité ni en statistiques. Durée : 6 h. Date : 17/09/2026.

Introduction : pourquoi ce cours ?

Bienvenue dans ce bootcamp. En une journée, vous allez toucher du doigt deux métiers très recherchés du numérique :

- **La cybersécurité** (et plus précisément le *test d'intrusion*, ou « pentest ») : le métier de celles et ceux qui, avec autorisation, jouent le rôle de l'attaquant pour trouver les failles d'un système **avant** les vrais pirates.
- **L'expérimentation par les données** (le *test A/B*) : la méthode que les entreprises du web utilisent pour décider, chiffres à l'appui, si un changement (un bouton, un texte, un prix) améliore vraiment les résultats.

 **À retenir** Ces deux mondes ont un point commun essentiel : **les données**. La cybersécurité protège les données ; l'AB testing les exploite pour décider. Un test A/B repose sur des données d'utilisateurs — souvent personnelles. Si ces données ne sont pas protégées, l'expérience entière perd sa fiabilité et sa légalité. Sécurité et données sont les deux faces d'une même pièce.

Comment lire ce support ? Ce document explique *tout ce qu'il faut comprendre*. Les manipulations pas-à-pas, elles, se trouvent dans votre **Cahier de TP** séparé. Ici, on prend le temps d'expliquer le *pourquoi*. Les encadrés colorés vous aident à repérer l'essentiel :

 **Astuce** — un conseil pratique.

⚠ **Attention** — un point de vigilance à ne pas rater.

🔑 **À retenir** — l'idée clé de la section.

⚖ **Cadre légal** — ce que dit la loi.

🛡 **Remédiation** — comment se protéger de l'attaque décrite.

MODULE 1 — Fondamentaux de la cybersécurité

1.1 La triade CIA : les trois piliers de la sécurité

Toute la cybersécurité tient dans trois mots. En anglais on parle de la **triade CIA** — rien à voir avec l'agence de renseignement : c'est l'acronyme de **Confidentiality, Integrity, Availability**. En français : **Confidentialité, Intégrité, Disponibilité**.

Pilier	Question posée	Exemple concret de ce qu'on veut éviter
Confidentialité	<i>Qui a le droit de voir l'information ?</i>	Une fuite de données : la liste des clients d'un site (emails, mots de passe) se retrouve en vente sur internet.
Intégrité	<i>L'information est-elle exacte et non modifiée ?</i>	Un défacement : un pirate remplace la page d'accueil d'un site officiel par son propre message. On ne peut plus faire confiance au contenu.
Disponibilité	<i>Le service est-il accessible quand on en a besoin ?</i>	Une attaque DDoS : le site est noyé sous les requêtes et devient inaccessible aux vrais clients.

Une analogie simple : votre boîte aux lettres.

- *Confidentialité* : seul vous avez la clé, personne d'autre ne lit votre courrier.
- *Intégrité* : personne ne peut ouvrir une lettre, la modifier, et la refermer sans que ça se voie.
- *Disponibilité* : le facteur peut toujours y déposer le courrier ; la serrure n'est pas bloquée à la glu.

🔑 **À retenir** Sécuriser un système, c'est protéger **en même temps** ces trois piliers. Une attaque vise souvent l'un d'eux en particulier — savoir lequel aide à comprendre le danger et la parade.

1.2 Le vocabulaire essentiel

Quelques mots reviennent en permanence. Voici le lexique de base :

Terme	Définition simple	Image mentale
Vulnérabilité	Une faiblesse dans un système (un bug, une mauvaise configuration, un mot de passe faible).	Une fenêtre laissée entrouverte.
Exploit	Le « mode d'emploi » ou le bout de code qui profite d'une vulnérabilité pour agir.	Le pied-de-biche qui force la fenêtre.
Payload (charge utile)	Ce que l'attaquant fait exécuter une fois entré (voler des données, ouvrir un accès...).	Ce que le cambrioleur emporte ou installe une fois dedans.
Surface d'attaque	L'ensemble de tous les points par lesquels on pourrait entrer (ports ouverts, formulaires, comptes...).	Toutes les portes et fenêtres de la maison.
Vecteur d'attaque	Le chemin précis réellement emprunté pour l'attaque.	La fenêtre par laquelle on est effectivement passé.
CVE	<i>Common Vulnerabilities and Exposures</i> : l'identifiant mondial et unique d'une vulnérabilité connue (ex. CVE-2021-44228).	Le « numéro de série » officiel d'une faille répertoriée.

💡 **Astuce** Retenez la différence **surface** (tous les points d'entrée possibles) vs **vecteur** (celui réellement utilisé). Réduire la surface d'attaque — fermer les portes inutiles — est la première mesure de défense.

1.3 Les grandes familles de menaces

Menace	En deux mots	Exemple
Ingénierie sociale / Phishing	Manipuler <i>l'humain</i> , pas la machine.	Un faux mail « votre banque » qui vous pousse à saisir votre mot de passe sur un site pirate.
Malware (logiciel malveillant)	Un programme conçu pour nuire (virus, rançongiciel...).	Un rançongiciel qui chiffre vos fichiers et réclame une rançon.
Injection SQL	Détourner la base de données via un champ de saisie.	Taper une astuce dans un champ « identifiant » pour lire toute la table des utilisateurs.
XSS (Cross-Site Scripting)	Injecter du code qui s'exécutera dans le navigateur d'autres visiteurs.	Poster un commentaire piégé qui vole les sessions de ceux qui le lisent.
Brute force	Essayer des milliers de mots de passe jusqu'à trouver le bon.	Un programme qui teste <code>123456</code> , <code>password</code> , <code>azerty</code> ... automatiquement.
Déni de service (DoS/DDoS)	Saturer un service pour le rendre indisponible.	Des milliers de machines qui bombardent un site jusqu'à le faire tomber.

⚠ Attention La menace n° 1 en entreprise n'est pas technique : c'est **le phishing**, qui vise l'humain. Aucune technologie ne remplace la vigilance des utilisateurs.

1.4 La méthodologie d'un test d'intrusion

Un pentest ne se fait pas au hasard : il suit toujours les mêmes étapes, comme une recette. Voici la chaîne, illustrée par l'analogie du **cambricoleur méthodique** (qui, lui, agit *sans* autorisation — nous, nous en aurons une, voir 1.5).

Étape	Ce qu'on fait	Analogie du cambrioleur
1. Reconnaissance	Récolter un maximum d'informations sur la cible, sans y toucher.	Observer la maison, les horaires, les habitudes.
2. Scan	Sonder activement la cible pour trouver les points d'entrée (ports, services).	Faire le tour, tester quelles portes et fenêtres existent.
3. Énumération	Détailler chaque point trouvé (versions logicielles, comptes, dossiers).	Repérer le modèle exact de chaque serrure.
4. Exploitation	Utiliser une vulnérabilité pour entrer réellement.	Forcer la fenêtre repérée comme fragile.
5. Post-exploitation	Une fois entré : évaluer ce qu'on peut atteindre, garder l'accès.	Fouiller, voir jusqu'où on peut aller dans la maison.
6. Rapport	Documenter les failles trouvées et comment les corriger .	<i>(Le pentesteur, lui, rédige un compte rendu pour aider à se protéger.)</i>

 **À retenir** La grande différence entre un pentesteur et un pirate est **l'étape 6** : le pentesteur travaille pour *défendre*. Son livrable, c'est le rapport qui permet de corriger les failles. Aujourd'hui, vos « fiches de vulnérabilité » sont votre premier rapport de pentest.

1.5 Le cadre légal : la règle d'or absolue

C'est le point le plus important de la journée.

 **Cadre légal** En France, accéder ou se maintenir dans un système informatique sans autorisation est un **délit**, puni par les **articles 323-1 et suivants du Code pénal** (jusqu'à plusieurs années de prison et de fortes amendes). Le simple fait de *scanner* ou de *tenter* d'entrer sans droit est déjà répréhensible. Modifier ou supprimer des données aggrave la peine.

Un pentest **légal** repose sur trois conditions :

1. **Une autorisation écrite** du propriétaire du système (souvent un contrat, un « mandat »).

2. **Un périmètre défini** : la liste précise des machines/adresses que l'on a le droit de tester. Tout ce qui est en dehors est interdit.
3. **Des règles d'engagement** : ce qu'on a le droit de faire ou non (horaires, techniques autorisées...).

⚠ **Attention — Règle d'or du bootcamp** Tous les outils que vous manipulerez aujourd'hui (Nmap, Hydra, Burp Suite...) ne doivent **JAMAIS** être pointés vers un site ou un système réel, un ami, votre box internet, ou le réseau de l'école. **Uniquement les machines virtuelles créées en local sur votre ordinateur.** C'est justement pour cela que l'on travaille dans un réseau isolé (« host-only »). En dehors de ce cadre, vous commettez un délit.

MODULE 2 — Comprendre le pentest en pratique

Ce module explique le « pourquoi » des outils. Le « pas-à-pas » (les commandes à taper) est dans votre Cahier de TP.

2.1 Le réseau, les ports et les services

Avant d'attaquer, il faut comprendre comment deux machines se parlent.

- Chaque machine sur un réseau a une **adresse IP** (ex. **192.168.56.101**), comme une adresse postale.
- Sur une machine, différents **services** tournent en même temps : un serveur web, une base de données, un accès à distance... Pour les distinguer, chacun écoute sur un **port** numéroté (de 0 à 65535). Le port est comme un **numéro d'appartement** à une même adresse.
- **TCP** est le « protocole » qui garantit que les messages arrivent bien, dans l'ordre — comme un envoi en recommandé avec accusé de réception.

Quelques ports classiques :

Port	Service habituel	À quoi ça sert
21	FTP	Transfert de fichiers
22	SSH	Connexion à distance sécurisée
80	HTTP	Site web (non chiffré)
443	HTTPS	Site web (chiffré)
3306	MySQL	Base de données

🔑 **À retenir** Un **port ouvert** = une porte par laquelle on peut potentiellement dialoguer avec un service. Plus il y a de ports ouverts, plus la **surface d'attaque** est grande.

2.2 Lire un scan Nmap

Nmap est l'outil de référence pour la reconnaissance : il frappe à chaque porte (port) d'une machine et note lesquelles répondent, quel service se cache derrière, et souvent quelle **version** du logiciel.

Un résultat de scan ressemble à ceci :

```

PORT      STATE  SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu
80/tcp    open  http     Apache httpd 2.4.18
3306/tcp  open  mysql    MySQL 5.7.12

```

On y lit : le **port**, son **état** (**open** = ouvert), le **service** deviné, et sa **version**. La version est précieuse : connaître « Apache 2.4.18 » permet de chercher les **CVE** (failles connues) de cette version précise.

💡 **Astuce** Les options que vous utiliserez : **-sV** (détecte les **versions**), **-sC** (lance des **scripts** d'analyse standard), **-p-** (teste **tous** les ports). **nmap -sn** (avec un **n**) fait juste un « qui est là ? » sur le réseau, sans scanner les ports.

2.3 Le proxy d'interception (Burp Suite)

Quand votre navigateur parle à un site web, il envoie des **requêtes HTTP** invisibles. Un **proxy d'interception** comme **Burp Suite** se place *entre* votre navigateur et le site : il **met en pause** chaque requête pour que vous puissiez la lire, la comprendre, et même la **modifier** avant de l'envoyer.

Pourquoi c'est utile ?

- **Comprendre** : voir exactement quelles données partent (identifiant, mot de passe, paramètres cachés).
- **Tester** : modifier une valeur à la volée pour voir comment le serveur réagit — c'est ainsi qu'on découvre beaucoup de failles.

💡 **Astuce** Un site web « fait confiance » trop facilement à ce que lui envoie le navigateur. Le proxy montre que **tout** ce qui vient du navigateur peut être trafiqué. D'où la règle des développeurs : « *ne jamais faire confiance aux données envoyées par l'utilisateur* ».

2.4 L'injection SQL, expliquée en profondeur

C'est **la** faille à comprendre aujourd'hui. Voyons pourquoi elle marche.

Derrière un formulaire de connexion, le site pose une **question à sa base de données** (une requête SQL). Par exemple, pour afficher les infos d'un utilisateur dont on a saisi l'identifiant **1**, le code construit souvent la requête ainsi :

```
SELECT prenom, nom FROM utilisateurs WHERE id = '1';
```

Le problème : le **1** vient directement de **ce que l'utilisateur a tapé**, copié tel quel dans la requête. Que se passe-t-il si, au lieu de **1**, on tape la charge suivante ?

```
1' OR '1'='1
```

La requête construite devient alors :

```
SELECT prenom, nom FROM utilisateurs WHERE id = '1' OR '1'='1';
```

Décortiquons la condition `WHERE id = '1' OR '1'='1'` :

- `id = '1'` : vrai seulement pour l'utilisateur 1.
- `OR` : « ou bien... ».
- `'1'='1'` : cette comparaison est **toujours vraie** (1 est toujours égal à 1 !).

Comme on relie les deux par `OR`, la condition entière est **toujours vraie pour chaque ligne**. Résultat : la base renvoie **tous les utilisateurs**, pas seulement le n° 1. L'attaquant a détourné la logique de la requête en injectant un morceau de code SQL dans un champ prévu pour une simple donnée.

Schéma avant / après :

	Condition évaluée	Ce que renvoie la base
Saisie normale <code>1</code>	<code>id = '1'</code>	La ligne de l'utilisateur 1
Charge injectée <code>1' OR '1'='1'</code>	<code>id = '1' OR '1'='1'</code> (toujours vrai)	Toutes les lignes

 **Remédiation** On ne corrige **pas** en interdisant l'apostrophe (contournable). La vraie solution : les **requêtes préparées** (*prepared statements* / requêtes paramétrées). Le code sépare alors la **structure** de la requête des **données** : la saisie est traitée comme une simple valeur, jamais comme du code exécutable. En complément : le principe du **moindre privilège** (le compte de la base n'a que les droits strictement nécessaires) et la **validation des entrées**.

2.5 Le XSS (Cross-Site Scripting reflected)

Le XSS, c'est l'injection SQL... mais côté **navigateur**. Au lieu de piéger la base de données, on piège la **page web** pour qu'elle exécute du code (JavaScript) chez les visiteurs.

Dans un champ de recherche vulnérable, on tape :

```
<script>alert('test')</script>
```

Si le site réaffiche ce texte **sans le neutraliser**, le navigateur ne voit pas un texte : il voit une **instruction** et l'exécute — ici, une petite fenêtre **alert**. Anodin en démonstration, mais un vrai attaquant remplace le **alert** par du code qui **vole les cookies de session** (pour se faire passer pour la victime), redirige vers un site pirate, ou modifie la page.

Pourquoi « reflected » (réfléchi) ? Parce que le code injecté est renvoyé (« réfléchi ») immédiatement dans la réponse, souvent via un lien piégé qu'on envoie à la victime.

 **Remédiation Échapper** systématiquement les données affichées (transformer `<` en `<`, etc.) pour qu'elles soient traitées comme du texte, jamais comme du code. Ajouter une **CSP** (*Content Security Policy*), un en-tête qui indique au navigateur quels scripts il a le droit d'exécuter. Marquer les cookies sensibles **HttpOnly** pour qu'un script ne puisse pas les lire.

2.6 Le brute force et les mots de passe

Le **brute force** (force brute) consiste à essayer un très grand nombre de mots de passe jusqu'à trouver le bon. On n'y va pas au hasard : on utilise une **wordlist**, un immense fichier de mots de passe fréquents. La plus célèbre, **rockyou.txt** (livrée avec Kali), contient environ 14 millions de mots de passe issus de fuites réelles. Autant dire que **motdepasse** ou **azerty123** y figurent en bonne place.

L'outil **Hydra** automatise l'envoi de ces milliers d'essais à un formulaire de connexion.

 **Remédiation** Trois protections complémentaires :

- **Politique de mot de passe robuste** : long, imprévisible (une phrase de passe vaut mieux qu'un mot compliqué court). Un mot de passe absent des wordlists ne sera pas deviné.

- **Rate-limiting / verrouillage** : bloquer ou ralentir après quelques essais ratés. Le brute force devient alors trop lent pour aboutir.
- **MFA** (authentification multifacteur) : même avec le bon mot de passe, il manque le second facteur (code sur le téléphone). C'est la protection la plus efficace.

🔑 **À retenir** Toutes les failles de ce module ont la même racine : **le système fait trop confiance à ce qui vient de l'extérieur** (une saisie, un mot de passe, une requête). Bien défendre, c'est **ne jamais faire confiance par défaut** et toujours vérifier.

MODULE 3 — L'AB Testing : décider avec les données

3.1 Qu'est-ce qu'un test A/B ?

Imaginez un site de vente en ligne. L'équipe se demande : « *Notre bouton d'achat convertit-il mieux en **rouge** ou en **vert** ?* » Plutôt que de trancher au feeling, on fait une **expérience** :

- On montre le bouton **rouge** (version **A**, dite « contrôle ») à la moitié des visiteurs, tirés au sort.
- On montre le bouton **vert** (version **B**, dite « variante ») à l'autre moitié.
- On mesure pour chaque groupe une **métrique** : ici le **taux de conversion** (le pourcentage de visiteurs qui achètent).

Si le groupe B achète nettement plus, on adopte le vert. Un test A/B, c'est donc une **expérience contrôlée** : on ne change qu'**une seule chose** à la fois, et le tirage au sort garantit que les deux groupes sont comparables.

⚠ **Attention** Le tirage au sort (**randomisation**) est le cœur de la méthode. C'est lui qui assure que la seule différence entre A et B est le changement testé — pas l'âge, l'heure, ou l'appareil des visiteurs.

3.2 Les hypothèses : H0 et H1 (l'analogie du procès)

Avant de regarder les chiffres, on pose deux hypothèses opposées :

- **H0** (hypothèse *nulle*) : « Il n'y a *AUCUNE* différence entre A et B. » Le changement ne sert à rien.
- **H1** (hypothèse *alternative*) : « B change vraiment la métrique. »

Le raisonnement est celui d'un **procès** :

- **H0 = la présomption d'innocence.** On part du principe que le changement est « innocent » (sans effet) tant qu'on n'a pas de preuve du contraire.
- Les **données** sont les preuves apportées au tribunal.
- On ne « condamne » (on ne rejette H0) que si les preuves sont **suffisamment fortes**. Sinon, on acquitte « au bénéfice du doute » — ce qui ne prouve pas l'innocence, juste qu'on manque de preuves.

🔑 **À retenir** On ne *prouve* jamais H0. On peut seulement **la rejeter** (preuve d'un effet) ou **ne pas réussir à la rejeter** (preuve insuffisante). « Pas de différence significative » veut dire « *on n'a pas assez de preuves* », pas « *les deux sont identiques* ».

3.3 La p-value et le seuil de significativité

La **p-value** est le chiffre central du test. Sa définition, à lire deux fois :

La p-value est la **probabilité d'observer un écart au moins aussi grand** que celui mesuré entre A et B, **si en réalité H0 était vraie** (c'est-à-dire s'il n'y avait aucune différence).

Autrement dit : « Si le changement ne servait à rien, quelle serait la probabilité d'obtenir un résultat aussi tranché par le seul hasard ? »

- Une p-value **petite** (proche de 0) = un tel écart serait très **improbable** par pur hasard → c'est un **indice fort** que la différence est réelle → on **rejette H0**.

- Une p-value **grande** = l'écart observé pourrait facilement venir du hasard → on **ne rejette pas H0**.

On fixe à **l'avance** un seuil de décision, noté α (alpha). La convention universelle est :

$$\alpha = 0,05 \quad (\text{soit } 5 \%)$$

La règle de décision :

- Si **p-value < 0,05** → différence **statistiquement significative** → on rejette H0.
- Si **p-value ≥ 0,05** → différence **non significative** → on ne rejette pas H0.

⚠ **Attention** — ce que « **significatif** » ne veut **PAS** dire « Statistiquement significatif » signifie seulement « *l'écart est probablement réel, pas dû au hasard* ». Cela **ne dit rien** sur la **taille** de l'effet ni sur son **intérêt business**. Un écart minuscule (10,0 % vs 10,1 %) peut devenir « significatif » avec assez de monde, sans valoir la peine d'être déployé. Toujours regarder **et** la significativité **et** l'ampleur réelle.

3.4 Taille d'échantillon et puissance

Combien de visiteurs faut-il ? Cela dépend de la **finesse** de ce qu'on cherche à détecter.

- Détecter un **gros** écart (10 % → 20 %) demande **peu** de monde.
- Détecter un **petit** écart (10 % → 10,5 %) demande **beaucoup** de monde.

La **puissance statistique** est la capacité du test à **détecter un effet qui existe réellement**. Plus l'échantillon est grand, plus la puissance est élevée, et plus on est capable de repérer de petites différences vraies.

💡 **Astuce (intuition à retenir)** Avec **peu** de données, même une vraie différence peut passer inaperçue (p-value trop grande). Avec **beaucoup** de données, on détecte des différences de plus en plus fines. En TP, vous ferez varier la taille des groupes et verrez la p-value bouger : c'est cette intuition qu'il faut ancrer.

3.5 L'intervalle de confiance à 95 %

Quand on mesure « 12 % de conversion », ce n'est qu'une **estimation** sur un échantillon : la « vraie » valeur est un peu au-dessus ou en-dessous. L'**intervalle de confiance à 95 %** donne une **fourchette** plausible, par exemple [11,1 % ; 12,9 %].

Interprétation correcte : « avec la méthode utilisée, ce type d'intervalle contient la vraie valeur dans 95 % des cas. »

💡 **Astuce** Un usage très pratique : si les intervalles de A et de B **ne se chevauchent pas**, c'est un signe fort que la différence est réelle. S'ils se recouvrent largement, méfiance.

3.6 Quel test choisir : Z sur proportions ou Khi-deux ?

Quand la métrique est un **oui/non** (a converti / n'a pas converti), deux tests classiques répondent à la même question :

Test	Quand l'utiliser	Ce qu'il compare
Test Z sur proportions	Comparer 2 groupes (A vs B) sur un taux (oui/non).	Directement les deux proportions. Donne aussi le sens de l'écart.
Test du Khi-deux (χ^2)	Comparer 2 groupes ou plus , ou plusieurs catégories.	La répartition observée vs celle attendue sous H0.

Pour un A/B classique à deux variantes et une métrique oui/non, le **test Z sur proportions** est le plus direct — c'est celui du TP. Les deux donnent des conclusions cohérentes sur un cas 2x2.

3.7 Les pièges classiques à éviter

Piège	En quoi c'est faux	La bonne pratique
Peeking / arrêt anticipé	Regarder les résultats en continu et s'arrêter dès que « $p < 0,05$ » gonfle artificiellement les faux positifs.	Fixer à l' avance la taille d'échantillon et la durée ; attendre la fin.
Échantillon trop petit	Trop peu de données = test peu puissant = on rate les vraies différences.	Calculer la taille nécessaire avant de lancer.
Trop de variantes en même temps	Tester 20 variantes garantit d'en trouver une « significative » par pur hasard.	Limitier le nombre de tests ; corriger le seuil si besoin.
p-hacking	Trafiquer l'analyse (filtres, sous-groupes) jusqu'à obtenir $p < 0,05$.	Définir la métrique et l'analyse avant de voir les données.

 **À retenir** Un test A/B honnête se **conçoit avant** de collecter les données : hypothèse, métrique, taille d'échantillon et seuil sont fixés d'avance. Décider après coup, c'est se tromper soi-même.

3.8 Le lien cybersécurité données

Un test A/B manipule des **données d'utilisateurs réels** : qui a vu quoi, qui a cliqué, souvent reliées à un identifiant. C'est là que les deux mondes de la journée se rejoignent.

 **Cadre légal (RGPD)** En Europe, le **RGPD** encadre le traitement des données personnelles : finalité définie, minimisation (ne collecter que le nécessaire), et protection des données. Un test A/B doit donc **anonymiser** ou **pseudonymiser** les données autant que possible.

Et sur le plan sécurité :

- Une base de résultats A/B **mal protégée** est une **surface d'attaque** de plus : un attaquant pourrait lire des données utilisateurs sensibles.
- Pire, un attaquant qui **modifierait** les résultats (atteinte à l'**intégrité**, pilier CIA !) fausserait la décision business — on déploierait le mauvais choix en toute confiance.

🔑 **À retenir** La fiabilité d'une expérimentation dépend de la **sécurité** des données qui la nourrissent. Confidentialité et intégrité (deux piliers du Module 1) sont les conditions d'un A/B testing digne de confiance.

Glossaire

Terme	Définition courte
Triade CIA	Confidentialité, Intégrité, Disponibilité : les 3 objectifs de la sécurité.
Vulnérabilité	Faiblesse exploitable d'un système.
Exploit	Code/technique qui profite d'une vulnérabilité.
Payload	Ce que l'attaquant fait exécuter une fois entré.
Surface d'attaque	Ensemble des points d'entrée possibles.
Vecteur d'attaque	Le chemin réellement emprunté.
CVE	Identifiant mondial unique d'une faille connue.
Port	Numéro identifiant un service sur une machine (0–65535).
Nmap	Outil de scan réseau (ports, services, versions).
Proxy d'interception	Outil (Burp) qui lit/modifie les requêtes HTTP.
Injection SQL	Détournement d'une requête base de données via une saisie.
Requête préparée	Parade à l'injection SQL : sépare code et données.
XSS	Injection de code exécuté dans le navigateur des visiteurs.
CSP	En-tête limitant les scripts autorisés (parade XSS).
Brute force	Essai massif de mots de passe.
Wordlist / rockyou	Fichier de mots de passe fréquents.
MFA	Authentification à plusieurs facteurs.
Test A/B	Expérience comparant deux versions (A contrôle, B variante).
H0 / H1	Hypothèse « aucun effet » / « effet réel ».
p-value	Probabilité d'un tel écart si H0 était vraie.
α (alpha)	Seuil de décision, en général 0,05.
Significatif	p-value < seuil : écart probablement réel ($\neq$ important).
Puissance	Capacité à détecter un effet réel.
Intervalle de confiance	Fourchette plausible de la vraie valeur (souvent 95 %).
Test Z / Khi-deux	Tests comparant des proportions.
RGPD	Règlement européen sur les données personnelles.

Pour aller plus loin

- **OWASP Top 10** — la liste de référence des 10 vulnérabilités web les plus courantes ; le point de départ de tout défenseur web.
- **TryHackMe** — plateforme d'entraînement au pentest, avec des parcours guidés pour débutants et sans rien à installer.
- **Documentation Kali Linux** — pour approfondir les outils utilisés aujourd'hui (Nmap, Hydra, Burp Suite).
- **DVWA sur GitHub** — le code et la doc de l'application vulnérable que vous attaquez en TP.
- **statsmodels — proportions_ztest** — la documentation du test statistique utilisé dans le notebook.
- **Optimizely — Guide A/B Testing** — la vision produit/business du test A/B, en complément de l'angle statistique de ce cours.

Fin du support de cours. Gardez-le à portée de main pendant les TP : chaque manipulation du Cahier de TP renvoie à une notion expliquée ici.